# A Mobile Phone based Voice-Controlled UGV

Divyansh Maheshwari, Shriyansh Chawda and GVV Sharma
Department of Electrical Engineering
Indian Institute of Technology, Hyderabad
Kandi, India 502284
gadepall@ee.iith.ac.in

***Abstract*—This paper implements a cloud-free voice-controlled Unmanned Ground Vehicle (UGV), consisting of a simple speech recognition engine and microcontroller based mobility. The novelty of this work lies in the fact that all voice commands are processed on an Android smartphone where all speech recognition algorithms are implemented. The key algorithms used are Mel-Frequency Cepstral Coefficient (MFCC) features combined with Dynamic Time Warping (DTW). User adaptation is supported through up to ten recorded templates per command, achieving 90–95% accuracy across 10 speakers and demonstrating language-agnostic operation in English, Hindi, Gujarati, and Tamil.**

***Index Terms*—UGV, voice control, MFCC, DTW, ESP32, BLE, Android, speech recognition, embedded systems**

## I. Introduction

Speech-based control removes the need for physical controllers and lowers the barrier for non-expert operation of ground robots. Most prior implementations use recognition to cloud automatic speech recognition (ASR) services [1], which introduces round-trip latency and ties the system to network availability. Running a full speech model on the robot microcontroller directly is equally impractical: a device such as the ESP32 carries insufficient flash and RAM for any modern recogniser.

This paper resolves the compute–connectivity dilemma by offloading all recognition to the Android smartphone that the operator already carries. Short utterances are converted to MFCC feature matrices [2] and matched against stored templates using DTW [3]. Both steps run entirely on the phone, so no audio ever leaves the device. Neural keyword-spotting alternatives exist for microcontrollers [4], [5] but demand labelled training corpora and quantisation pipelines that a DTW system avoids entirely. A 7-byte bluetooth low energy (BLE) command string is transmitted to the ESP32 only after a match is confirmed, reducing the wireless payload by over 99% compared to streaming raw audio [6].

### A. Key Contributons

The specific contributions of this work are

1) A smartphone-resident MFCC+DTW pipeline that operates without any cloud or internet dependency.
2) A Best-of-$N$ template strategy allowing each user to enrol up to ten personal utterances per command for speaker adaptation.
3) A latency-optimised BLE scheme that cuts end-to-end response time from 2–4 s to approximately 1.6 s by transmitting only the recognised command label.

## II. System Architecture

### A. Hardware Setup

Table I lists the various hardware components for UGV control. Fig. 1 shows the connections between the ESP32 and the UGV motor driver. Table II shows how the motor driver controls UGV mobility.

TABLE I
List of Components

| Component | Qty. | Description |
|---|---|---|
| Chassis frame | 1 | Holds all components |
| Side wheels (DC motors) | 2 | Drive wheels |
| Caster wheel | 1 | Stability and smooth turning |
| ESP32 dev board | 1 | Microcontroller + BLE |
| L293D motor driver IC | 1 | Drives DC motors |
| DC motors (TT gear) | 2 | Propulsion |
| Breadboard | 1 | Circuit connections |
| Jumper wires | 11 | Electrical connections |
| Power bank / LiPo | 1 | Portable power supply |
| Android phone (API 23+) | 1 | Recognition + BLE central |
| Micro-USB / USB-C cable | 1 | ESP32 flashing |

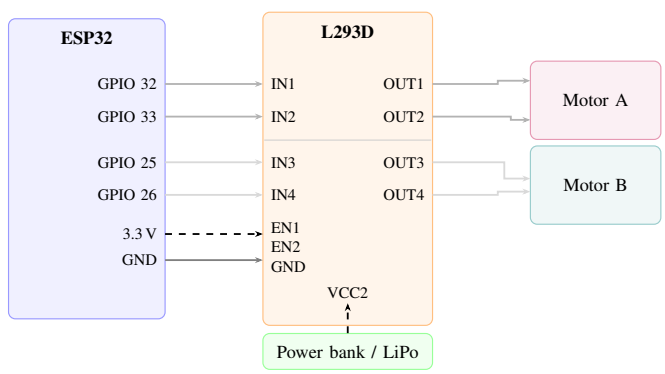


Fig. 1. Wiring schematic: ESP32 to L293D to DC motors.

TABLE II
MOTOR DIRECTION TRUTH TABLE

| IN1 | IN2 | IN3 | IN4 | Result |
|---|---|---|---|---|
| H | L | H | L | Move Forward |
| L | H | L | H | Move Back |
| L | H | H | L | Turn Left |
| H | L | L | H | Turn Right |
| L | L | L | L | Motor Stop |

### B. Software Overview

Two hardware nodes form the complete system: an Android smartphone that handles all audio capture and recognition, and an ESP32 microcontroller that handles BLE reception and motor actuation. Fig. 2 shows the key functions executed by the smartphone and the ESP32.

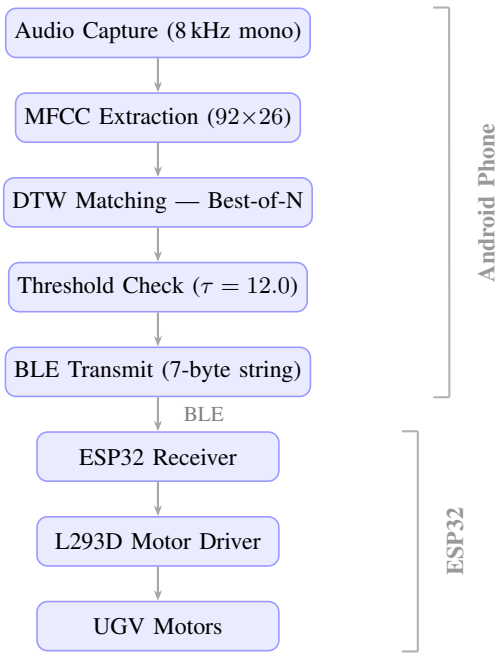


Fig. 2. System pipeline block diagram.

## III. SIGNAL PROCESSING

Each 1.5 s audio recording captured at 8 kHz is converted into a compact $92 \times 26$ feature matrix using Mel-Frequency Cepstral Coefficients (MFCC). The pipeline frames the signal into 92 overlapping windows, applies a Hamming window and FFT to extract spectral energy, maps it onto a 26-band Mel filterbank that mimics human auditory perception, and decorrelates the result via DCT-II into 13 cepstral coefficients. First-order delta coefficients are appended to capture temporal dynamics, expanding each frame to 26 dimensions. Finally, Cepstral Mean and Variance Normalisation (CMVN) [7] removes speaker-specific biases by standardising each coefficient across all 92 frames, making the features robust across different voices and microphones.

## IV. PATTERN MATCHING

The MFCC feature matrix of a spoken query is compared against stored command templates using Dynamic Time Warping (DTW) [3]. DTW computes a frame-by-frame Euclidean distance and finds the optimal alignment path between the query and each template, allowing for natural variations in speaking speed. The accumulated path cost is normalised by the path length to remove duration bias. A Best-of-$N$ strategy then takes the minimum score across all stored templates (up to $N = 10$ per command), so the system always matches against the user's closest recorded utterance. A command is accepted only if its winning score falls below threshold $\tau = 12.0$; otherwise the input is rejected as noise or an unknown word.

## V. SOFTWARE IMPLEMENTATION

### A. ESP32 Firmware

Three source files make up the firmware, built with PlatformIO on the Arduino framework.

**ble_receive.cpp** sets up a BLE GATT server with a WRITE characteristic (`UUID: ...9002`) for incoming command strings and a NOTIFY characteristic (`UUID: ...9003`) for acknowledgements. Any string not matching the five known commands is discarded. Advertising for new connection restarts automatically on disconnect.

**motor.cpp** toggles four GPIO pins (32, 33, 25, 26) to drive the L293D. No PWM is used.

**main.cpp** implements the command dispatcher:

Listing 1. ESP32 command dispatcher (main.cpp)

```
void loop() {
char cmd[16] = {0};
if (!ble_receive_command(cmd, sizeof(cmd)))
{ delay(5); return; }

if (!strcmp(cmd,"forward")) { motor_forward(); }
else if (!strcmp(cmd,"back")) { motor_back(); }

else if (!strcmp(cmd,"left"))
{ motor_left(); delay(600); motor_stop(); }

else if (!strcmp(cmd,"right"))
{ motor_right(); delay(600); motor_stop(); }

else if (!strcmp(cmd,"stop")) { motor_stop(); }}
```

*forward* and *back* run until a *stop* is received. *left* and *right* run for 600 ms then auto-stop, giving approximately a 90° turn. The duration is adjustable for different chassis.

### B. Android Application

The app captures 1.5 s of audio at 8 kHz mono (16-bit PCM, 24 kB per utterance) using the `AudioRecord` API. The 8 kHz sample rate was chosen deliberately: speech energy is concentrated below 4 kHz, so the Nyquist criterion is satisfied while halving the data volume compared to 16 kHz, directly reducing FFT computation and template storage. All signal processing runs on a background thread to keep the

UI responsive; results are posted back to the UI thread on completion. A 200 ms pause before each `AudioRecord` initialisation prevents microphone release errors on back-to-back recordings.

When a match is confirmed (score $< \tau$), a 7-byte ASCII command string is written to the BLE GATT characteristic (`UUID: ...9002`), reducing the wireless payload by over 99% compared to streaming raw PCM. The ESP32 acknowledges receipt via a NOTIFY characteristic (`UUID: ...9003`), completing the control loop.

The app provides three screens. The *Main* screen presents five command buttons with live recognition feedback after each utterance. The *Debug* screen (Fig. 3) displays per-command DTW scores as a live bar chart, enabling the user to tune $\tau$ by observing the separation between correct and incorrect command scores. The *Training* screen lets users record up to 10 templates per command, showing per-command slot counters (e.g. 7/10) to track enrolment progress.

### C. Out-of-the-Box Deployment

Flashing the firmware and installing the APK is all that is required. The app scans for a BLE device named `ESP32_UGV` and connects automatically. Once paired, the user speaks a command and the vehicle responds within approximately 1.6 s.

## VI. User Adaptation and Tuning

### A. Recording Additional Templates

The `TemplateStore` always holds template index 0, which ships pre-built inside `assets` and was recorded by the original developer. Slots 1–9 are filled on the target device through `DebugActivity`'s record button; no PC or cable is needed after the initial APK install. How many slots to fill depends on the use case — Table III gives concrete guidance.

TABLE III
Recommended Template Count per Scenario

| Scenario | Min. | Recommended |
|---|---|---|
| Quick prototype, quiet room | 1 | 3–5 |
| Daily driver, single speaker | 5 | 8–10 |
| Noisy environment | 8 | 10 |
| Shared templates across speakers | Not supported | |

### B. Threshold Tuning

Picking a good $\tau$ is straightforward with the live score display in the debug screen. Run several genuine utterances and note the highest DTW score that still matches the correct command; then run a few wrong commands and note the lowest score that correctly rejects them. Set $\tau$ anywhere in the gap between those two numbers. In practice, 8–10 personal templates recorded in a quiet room push the correct-command scores down to the 3.0–7.5 range (Table IV), so a threshold of 8.0–10.0 gives comfortable separation for a single speaker.

### C. Turn Duration

Lateral movement is timed rather than angle-sensed: `main.cpp` holds the turn GPIO pattern for 600 ms before issuing a stop, which yields roughly 90° on the development chassis at full motor voltage. Heavier or slower builds need a longer delay; faster or lighter ones need a shorter one. The constant is a single integer literal in `main.cpp` and requires no recompilation of any Android code.

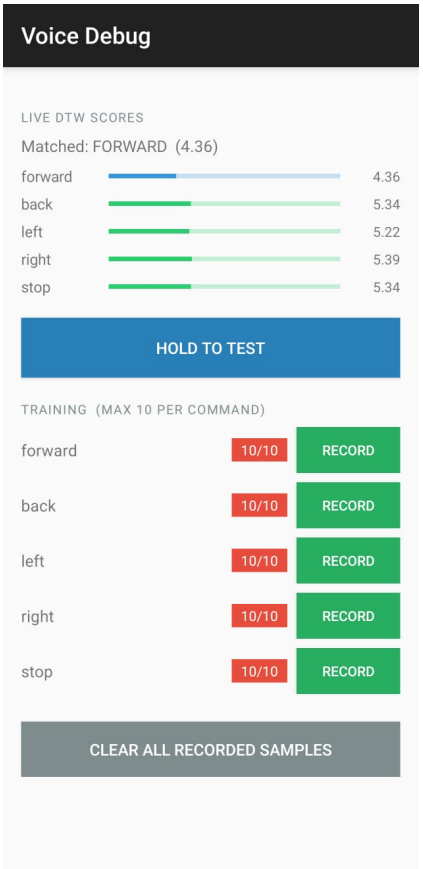


Fig. 3. Debug Activity screen showing live DTW scores after speaking *forward*. The blue bar indicates the winning command (score 4.36), while all other commands score above 5.0. All five commands have 10 recorded templates (10/10), producing well-separated scores and reliable recognition.

## VII. Results and Discussion

### A. Multi-Speaker Evaluation

Ten volunteers each recorded 10 templates per command through the in-app training interface, then independently spoke each command multiple times in a quiet indoor room. Across all speakers and all five commands the system maintained **90–95%** accuracy, as broken down in Table IV.

TABLE IV
Typical DTW Scores (single speaker, quiet environment)

| Condition | Score Range | Outcome |
|---|---|---|
| Correct command, 10 templates | 3.0 – 7.5 | Match ($< \tau$) |
| Correct command, 1 template | 5.0 – 11.0 | Match / borderline |
| Wrong command | 11.0 – 22.0 | No match ($> \tau$) |
| Silent / noise | 18.0 – 30.0 | No match |

DTW is fundamentally a speaker-dependent algorithm, so achieving this accuracy across 10 different voices warrants explanation. Two mechanisms are responsible. First, CMVN subtracts the per-coefficient mean and divides by the standard deviation computed over the 92-frame window, stripping out slow-varying spectral biases that differ between speakers. Second, the Best-of-N rule picks the single closest template out of up to 10 stored ones, so each user's personal recordings effectively anchor the matcher to their own vocal characteristics.

### B. Cross-Language Evaluation

A subset of participants issued commands in **Hindi, Gujarati, and Tamil** rather than English. Accuracy stayed within the 90–95% band across all three languages. No changes to the pipeline were needed: because `MFCCProcessor` works on raw acoustic energy and `DTWMatcher` compares 26-dimensional frame vectors, the system carries no phoneme tables, pronunciation dictionaries, or language models that would tie it to a specific language. Any consistently pronounced word maps to a repeatable MFCC trajectory and can therefore serve as a command.

### C. Latency

Button press to motor response measured at roughly 1.6 s. The previous ESP32-resident pipeline took 2–4 s because it had to receive all 24,000 bytes of 8 kHz PCM over BLE before extraction could begin. Shifting MFCC and DTW to the phone reduces the BLE transfer to a 7-byte ASCII string, and that single change accounts for most of the latency improvement. Table V summarises the key system-level trade-offs between the two architectures.

TABLE V
ESP32-RESIDENT VS PHONE-RESIDENT ARCHITECTURE

| Metric | ESP32-Resident | Phone-Resident |
|---|---|---|
| End-to-end latency | 2–4 s | $\sim$1.6 s |
| BLE payload per command | 24,000 bytes | 7 bytes |
| ESP32 flash used | $\sim$80 kB | $<$1 kB |
| ESP32 RAM (DTW matrix) | $O(F_1 F_2)$ | none |
| Cloud / internet required | No | No |

### D. Limitations

Each new speaker must record their own templates; there is currently no speaker-independent enrolment path. In noisy settings the `NoiseSuppressor` API offers some relief but does not fully recover accuracy. On the ESP32 side, the 600 ms fixed turn timer is a heuristic — it produces approximately 90° on the test chassis but needs manual adjustment for any build with different motor speed or chassis weight. Addressing all three points — speaker independence, noise robustness, and closed-loop turning — is left to future work.

## VIII. CONCLUSIONS

Relocating MFCC extraction and DTW matching from the ESP32 to an Android smartphone produces a fully on-device, cloud-free voice control system for a UGV. The shift cuts end-to-end latency from 2–4 s to 1.6 s and recovers 80 kB of microcontroller flash. CMVN normalisation and the Best-of-$N$ template strategy together yield 90–95% accuracy across 10 speakers and four languages — English, Hindi, Gujarati, and Tamil — despite DTW being inherently speaker-dependent.

Three limitations remain. Each new speaker must record personal templates, accuracy drops in noisy environments even with Android's `NoiseSuppressor` API active and Turn duration is not same for all UGV builds. Future work will investigate compact convolutional or recurrent keyword-spotting models that run on-device without per-user enrolment, alongside data augmentation for noise robustness and a larger-scale speaker evaluation.

## REFERENCES


[1] G. Li, S. Mei, H. Yang, X. Zhang, H. Wang, and F. Sun, "Speech interaction with robots: A review," *International Journal of Advanced Robotic Systems*, vol. 14, no. 6, 2017.

[2] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.

[3] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, 1978.

[4] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello edge: Keyword spotting on microcontrollers," in *arXiv preprint arXiv:1711.07128*, 2017.

[5] S. Choi, S. Seo, and B. Moon, "An efficient keyword spotting technique using a recurrent neural network for resource-constrained devices," *Applied Sciences*, vol. 9, no. 9, p. 1885, 2019.

[6] K. Mikhaylov and J. Tervonen, "Evaluation of power consumption for Bluetooth Low Energy," in *2013 IEEE 24th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC)*. IEEE, 2013, pp. 155–159.

[7] S. Furui, "Cepstral analysis technique for automatic speaker verification," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 29, no. 2, pp. 254–272, 1981.

[8] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.

[9] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[10] A. Maier *et al.*, "Design and implementation of ESP32-based IoT devices," *Sensors*, vol. 23, no. 15, p. 7023, 2023.

[11] J. Gonzalez *et al.*, "Implementation of a speech recognition system in a DSC," *IEEE Latin America Transactions*, vol. 14, no. 6, pp. 2657–2662, 2016.


## APPENDIX

This appendix provides step-by-step instructions for setting up the complete development environment, flashing the ESP32 firmware, and building and installing the Android application.

### A.1 Prerequisites

The following tools must be installed on the host PC before proceeding:

- **Python 3.8+** — required for template conversion and PlatformIO's build system. Available at https://www.python.org.

- **Git** — for cloning the repository. Available at https://github.com/gadepall/voice-ugv.git.
- **VS Code** (recommended) or any editor with PlatformIO support.

### A.2 ESP32 Firmware

1) **Install PlatformIO.** Install the PlatformIO IDE extension in VS Code, or install the PlatformIO Core CLI via:

```
pip install platformio
```

2) **Clone / extract the project.** Place the `ugv_esp32/` folder on your PC.
3) **Connect the ESP32** to the PC via USB.
4) **Build and flash** the firmware:

```
pio run --target upload
```

PlatformIO automatically installs the `ESP32 BLE Arduino` library declared in `platformio.ini`. After flashing, the ESP32 advertises itself as `ESP32_UGV` over BLE and awaits a connection.

5) **Verify** by opening the serial monitor (`pio device monitor`, 115200 baud) and confirming the message `BLE started, advertising...`.

### A.3 Android Application

*Option A — Install the pre-built APK (recommended):*

1) Copy `ugv_android.apk` to the Android device.
2) On the device, enable *Settings* → *Security* → *Install unknown apps* for your file manager.
3) Open the APK file and tap *Install*.

*Option B — Build from source with Android Studio:*

1) **Install Android Studio** (Hedgehog or later) from https://developer.android.com/studio. During setup, install the Android SDK (API level 33+) and the Android NDK if prompted.
2) **Open the project.** In Android Studio, choose *File* → *Open* and select the `ugv_android/` folder. Wait for Gradle sync to complete.
3) **Build the APK.** Select *Build* → *Build Bundle(s) / APK(s)* → *Build APK(s)*. The signed debug APK is placed in:

```
app/build/outputs/apk/debug/app-debug.apk
```

4) **Deploy to device.** Connect an Android phone (USB debugging enabled), then click *Run* → *Run 'app'* or use:

```
adb install app/build/outputs/apk/debug/app-
    debug.apk
```

### A.4 First-Time Setup on the Phone

1) Enable Bluetooth on the Android device.
2) Open the **UGV Controller** app. The *SetupActivity* wizard launches automatically on first run, guiding you through BLE pairing.
3) Tap *Connect to ESP32*. The app scans for a BLE device named `ESP32_UGV` and connects automatically.
4) Follow the on-screen prompts to record 5 templates per command (10 recommended for best accuracy).
5) Tap any command button to issue voice commands. The recognized command and DTW score are shown on the main screen; detailed per-command scores are available in the *Debug* screen.

### A.5 Troubleshooting

- **AudioRecord failed:** A 200 ms delay is already applied between recordings. If the error persists, close other apps using the microphone.
- **BLE not connecting:** Ensure the ESP32 is powered and advertising (`ESP32_UGV` appears in a BLE scanner app). Force-stop the UGV app and reconnect.
- **Poor recognition accuracy:** Record additional templates in a quiet environment. Reduce the threshold $\tau$ in `DTWMatcher.java` if valid commands are being rejected.
- **Gradle build errors:** Ensure JDK 17 is selected in *File* → *Project Structure* → *SDK Location*.